FEAT-ZD-48600 · DE-04 Identify the sources/pipelines to be changed (Ingestion) ■ · Components, layers, sources or pipelines to change

Data Engineering agent · item task.components · source hobasa-data-engineering-agent/outputs/FEAT-ZD-48600/task.components.md · approved by Data engineer sim.data_engineer

task.components: FEAT-ZD-48600

The sources and pipelines to create or update in the **ingestion layer** for the Data Engineering lane of FEAT-ZD-48600 (ZD-48600, "Show which overdue invoices belong to vendors that usually pay late"). This is DE-04, Identify the sources/pipelines to be changed (Ingestion), attempt 1. There is no gate. Automated checks, the independent judge and sampled human evaluation review this output (record:brief). DE-07 later extends this file for the transformation pipeline (data-engineering-steps).

Field	Value	Source
Trace ID	FEAT-ZD-48600	record:brief
Step	DE-04 Identify the sources/pipelines to be changed (Ingestion), attempt 1	record:brief
Inputs	feat.tasks (b0fd4d2), feat.design (722a98f), org.data_model (3b03f15), task.repo_access (aac590a)	record:brief; mcp:hobasa-context (context_list)
Background read	feat.clarifications (DES-03, section 3 CL-DE1 to CL-DE11); task.db_changes (DE-03); DE-02 task.secrets_scope	journey:feat.clarifications; journey:task.db_changes; file:hobasa-data-engineering-agent/outputs/FEAT-ZD-48600/task.secrets_scope.md
Code read	`src/ap_pipeline/__init__.py`, `ingest.py`, `transform.py`, `dq.py`, `pipeline.py`, `schedule.json`, `tests/test_pipeline.py` (tool view), plus `ci.py` and `data/sample_feed.csv`	tool:repo; file:ap_pipeline/ci.py; file:ap_pipeline/data/sample_feed.csv
Repository / branch	ap_pipeline, branch `feature/story`, write access confirmed; no Databricks or database	task.repo_access (section 1); journey:task.db_changes
Data Engineering tasks in scope	TSK-01 (STORY-01, optional `paid_date`) and TSK-02 (STORY-02, `delays_by_vendor`)	feat.tasks (section 1)
Result	**The ingestion layer needs no code change.** There is no new source and no new pipeline. The vendor invoice feed stays the only source, and the header-driven `parse_feed` already carries a v3 `paid_date` column as a row key. The smallest change that lets the optional column into the model is an additive update to `transform.to_model` (TSK-01). The second change is one new pure module, `delays.py` (TSK-02), which DE-07 details for the transformation pipeline. `pipeline.run`, `dq.quality_report` and `schedule.json` stay as they are. No secret is needed in any environment	tool:repo; feat.design (A2 HC-1 to HC-3, B1.1); feat.tasks (TSK-01, TSK-02); file:hobasa-data-engineering-agent/outputs/FEAT-ZD-48600/task.secrets_scope.md

1. Existing sources and pipelines (read from the repository)

ID	Element	Layer	What it does today	Source
EX-1	Vendor invoice feed (CSV with header)	Source	The only source. It enters as `csv_text`, a string handed to `pipeline.run`. The repository holds no file, URL, connector or credential for it. The checked-in sample `data/sample_feed.csv` has the header `invoice_id,vendor,due_date,amount,paid`, which is the v2 layout without `paid_date`	tool:repo (`pipeline.py`, `ingest.py`); file:ap_pipeline/data/sample_feed.csv; file:hobasa-data-engineering-agent/outputs/FEAT-ZD-48600/task.secrets_scope.md (section 2)
EX-2	`ingest.parse_feed(csv_text)`	Ingestion	`csv.DictReader` over `io.StringIO`. It returns one `dict` per row, keyed by the header text. It reads no column by name	tool:repo (`ingest.py`)
EX-3	`transform.to_model(rows)`	Ingestion → model boundary	A single `try` block per row builds `{"id", "vendor", "due_date", "amount", "paid"}`. On `KeyError`, `ValueError`, `InvalidOperation` or `AttributeError` it appends `{"row": r, "reason": type(e).__name__}` to `rejected`. It returns `(records, rejected)` and does not read `paid_date`	tool:repo (`transform.py`)
EX-4	`dq.quality_report(records, rejected)`	Data quality	`rows`, `loaded`, `rejected`, and the sorted set of `reject_reasons`	tool:repo (`dq.py`)
EX-5	`pipeline.run(csv_text)`	Pipeline (orchestration)	`to_model(parse_feed(csv_text))`, then `quality_report`. Returns `(records, report)` and writes nothing	tool:repo (`pipeline.py`)
EX-6	`schedule.json`	Schedule	`daily`, `02:00`, `Asia/Kolkata`	tool:repo (`schedule.json`)
EX-7	`__version__` in `src/ap_pipeline/__init__.py`	Metadata	`0.4.0`	tool:repo (`__init__.py`); task.repo_access (section 1)
EX-8	`tests/test_pipeline.py`	Tests	Three tests: `test_runs_maps_and_rejects`, `test_paid_flag`, `test_missing_column_rejected`. All use inline feeds with no `paid_date` column	tool:repo (`tests/test_pipeline.py`)

2. Source feed versions

Version	Columns	Status	Handling at ingestion	Source
Feed v2 (today)	`invoice_id`, `vendor`, `due_date`, `amount`, `paid`	Must still load	The row has no `paid_date` key	file:ap_pipeline/data/sample_feed.csv; feat.design (NF-4, B1.3); feat.tasks (TSK-01 goal "feed v2 still loads")
Feed v3 (new)	v2 columns plus optional `paid_date` (ISO date, empty when unpaid)	Not yet in production (PQ-6). It gates release, not coding	`parse_feed` passes `paid_date` through as a key. For a short row the key is `None`	org.data_model ("paid_date added in feed v3"); record:memory (journey requirement: "optional paid_date column (ISO date, empty when unpaid)"); feat.design (B1.3, Part C "External dependencies"); journey :feat.clarifications (CL-DE11)

Target model after ingestion (org.data_model): Invoice `id` (str), `vendor` (str), `due_date` (date), `amount` (Decimal, 2 dp), `paid` (bool), `paid_date` (date or None).

3. Change list (the outcome of this step)

#	Source / pipeline	Layer	Action	Minimum change	Why this is the minimum	Task	Source
IN-1	Vendor invoice feed (source)	Source	**No new source; no change**	None. The same CSV feed gains the optional `paid_date` column on the feed owner's side (v3)	`parse_feed` is header-driven, so the new column needs no source-side configuration in this repository	TSK-01	tool:repo (`ingest.py`); org.data_model; feat.design (A4)
IN-2	`ingest.parse_feed`	Ingestion	**No code change**	None	It already returns every header column as a key. A v2 row lacks `paid_date`, and a short v3 row carries `None`. `to_model` handles both with `r.get("paid_date")`. TSK-01 lists `ingest.py` under "Must not change"	TSK-01	tool:repo (`ingest.py`); feat.design (B1.1, B1.3); feat.tasks (TSK-01)

#	Source / pipeline	Layer	Action	Minimum change	Why this is the minimum	Task	Source
IN-3	`transform.to_model` (HC-1)	Ingestion → model boundary	**Update, additive**	Add constant `PAID_DATE_REASON = "paid_date"` and private helper `_parse_paid_date(raw)`. A missing, `None`, blank or whitespace-only value gives `None`. Otherwise the helper returns `date.fromisoformat(value.strip())` and raises `ValueError` when the value is invalid. Keep the existing `try` block exactly as it is. Only for a row that passed it, call `_parse_paid_date(r.get("paid_date"))`. On `ValueError`, append `{"row": r, "reason": PAID_DATE_REASON}` and add no record. Otherwise set `record["paid_date"]` and append the record	This is the one point where feed columns become model fields. Parsing inside the existing `try` would report `"ValueError"` instead of `'paid_date'` and fail AC-01.4. The signature and the `(records, rejected)` shape stay unchanged	TSK-01	feat.design (B1.2 steps 1 to 3, DD-6, DD-7); feat.tasks (TSK-01 notes 1 to 3); journey:feat.clarifications (CL-DE1, CL-DE2)
IN-4	`dq.quality_report`	Data quality	**No code change**	None	`reject_reasons` is the sorted set of each rejected row's `reason`, so `'paid_date'` appears on its own	—	tool:repo (`dq.py`); feat.design (HC-3)

#	Source / pipeline	Layer	Action	Minimum change	Why this is the minimum	Task	Source
IN-5	`pipeline.run`	Pipeline	**No code change**	None	Records gain `paid_date` inside `to_model`. `delays_by_vendor` is called by the caller (in this feature, the end-to-end harness), not by `pipeline.run`	—	tool:repo (`pipeline.py`); feat.design (HC-3, B5); feat.tasks (TSK-01 "Must not change")
IN-6	`schedule.json`	Schedule	**No change**	None	The feed still runs daily at 02:00 Asia/Kolkata. TSK-01 lists it under "Must not change"	—	tool:repo (`schedule.json`); feat.design (B1.1); feat.tasks (TSK-01)
IN-7	`delays.delays_by_vendor` (HC-2)	Derived data, downstream of ingestion	**Create** new module `src/ap_pipeline/delays.py`	One pure function, `delays_by_vendor(records: Iterable[Dict]) -> Dict[str, List[int]]`. It takes paid records with a `paid_date`, computes `(paid_date - due_date).days`, keeps values in feed order and vendors in first-seen order, keeps negative values, does no I/O and does not modify `records`. No `__all__` is added to `__init__.py`	New behaviour goes into a new function, as the repository rule requires. It is listed here for completeness; DE-07 owns its transformation detail	TSK-02	feat.design (HC-2, B1.2); feat.tasks (TSK-02); file: ap_pipeline/CLAUDE.md
IN-8	New source, pipeline, storage, connector or schema	—	**None created**	None	The design adds no repository, service, runtime, library or storage. DE-03 found no database change	—	feat.design (DD-1, A1); journey:task.db_changes (section 4)

#	Source / pipeline	Layer	Action	Minimum change	Why this is the minimum	Task	Source
IN-9	`data/sample_feed.csv`	Sample data	**No change**	None	It stays a v2 sample, which documents v2 compatibility. It is outside the TSK-01 and TSK-02 file budgets, and the tests use inline synthetic feeds	file:ap_pipeline/data/sample_feed.csv; feat.tasks (TSK-01, TSK-02 "Files in budget"); tool:repo (`tests/test_pipeline.py`)	

Rules carried for coding (DE-08):

- Do not add `strip()` to `due_date` parsing.
- `amount` stays `Decimal`, never `float`.
- Every existing key and value stays unchanged.
- An unpaid row with a valid `paid_date` is accepted and keeps the date (interim DD-7, OQ-B6).
- A row that is invalid in an existing field and in `paid_date` keeps today's reason (DD-6, TQ-10).
- Use invalid test values `"2026-13-01"` and `"not-a-date"` only, with no compact date forms.

Sources: feat.tasks (TSK-01 "Must not change", tests); feat.design (DD-6, DD-7); journey:feat.clarifications (CL-DE3, CL-DE4, CL-DE5, CL-DE8); record:memory (D-2026-044, L-011).

4. Files in the change budget

File	Change	Item	Task	Source
`src/ap_pipeline/transform.py`	Update (additive)	IN-3	TSK-01	feat.tasks (TSK-01); feat.design (B1.1)
`src/ap_pipeline/__init__.py`	Version step only, from `0.4.0`; the next number is for the lane to confirm	IN-3	TSK-01	feat.tasks (section 2); task.repo_access (section 1)
`tests/test_pipeline.py`	Add `test_paid_date_empty_is_none`, `test_feed_v2_without_paid_date_column`, `test_paid_date_parsed_for_paid_row`, `test_invalid_paid_date_rejected`, `test_existing_reason_wins` and `test_unpaid_row_keeps_paid_date`. The three existing tests stay unchanged	IN-3	TSK-01	feat.tasks (TSK-01 tests); feat.design (B1.5)
`src/ap_pipeline/delays.py`	New	IN-7	TSK-02	feat.tasks (TSK-02); feat.design (B1.1)
`tests/test_delays.py`	New: `test_delays_by_vendor_*`	IN-7	TSK-02	feat.tasks (section 2, TSK-02)

Files that must not change: `ingest.py`, `dq.py`, `pipeline.py`, `schedule.json` (feat.tasks TSK-01; feat.design B1.1). CI is `/usr/bin/python3 ci.py`. It compiles the code, runs the unit tests, requires coverage of at least 90.0 over `src/ap_pipeline`, and runs static checks for `print`, `eval`, `exec`, bare `except` and lines over 140 characters. feat.tasks section 2 adds that "Every error branch needs its own test" (file:ap_pipeline/ci.py; feat.tasks

section 2). Agent judgement: the tests listed above for TSK-01 and TSK-02 already cover the blank, valid and invalid `paid_date` paths and each `delays_by_vendor` filter.

5. Secrets access in all environments

Environment	Secrets required for ingestion	Granted	Gap	Source
Development (`feature/story`, local CI `/usr/bin/python3 ci.py`)	None	None	None	file:hobasa-data-engineering-agent/outputs/FEAT-ZD-48600/task.secrets_scope.md (sections 2, 3); task.repo_access (section 2)
Integration (`dev`)	None	None	None	file:hobasa-data-engineering-agent/outputs/FEAT-ZD-48600/task.secrets_scope.md (section 3)
Production (`main`)	None	None	None	file:hobasa-data-engineering-agent/outputs/FEAT-ZD-48600/task.secrets_scope.md (section 3)

The ingestion code takes the feed as an in-memory string and opens no connection. IN-3 and IN-7 are pure functions. Coding and tests use synthetic rows only (`SYN-INV-*`, "Synthetic Vendor <letter>"), and no production feed is read (tool:repo; feat.design NF-10, SE-4; feat.tasks TSK-08 notes). If releasing feed v3 later adds a fetch step with a credential, DE-02 re-runs on that re-entry trigger (file:hobasa-data-engineering-agent/outputs/FEAT-ZD-48600/task.secrets_scope.md, section 5).

6. Dependencies and risks

ID	Item	Handling	Source
R-I1	Feed v3 is not yet in production (PQ-6). ZD-48470 brings feed v2	Gates release, not coding. Build on synthetic v3 rows. v2 keeps loading because `paid_date` is optional	feat.design (Part C "External dependencies"); feat.tasks (TSK-01 "Depends on"); journey:feat.clarifications (CL-DE11)
R-I2	`paid_date` parsed inside the existing `try` would report `"ValueError"` instead of `'paid_date'`	IN-3 parses it only after the `try` succeeds. `test_existing_reason_wins` and `test_invalid_paid_date_rejected` guard both orders	feat.design (B1.2, DD-6); journey:feat.clarifications (CL-DE1)
R-I3	`parse_feed` keys rows by the exact header text. A v3 header spelled differently from `paid_date` (case or spaces) would load silently as `paid_date = None` and the row would not be rejected	No code change. The column name is taken from org.data_model and the design. Confirm the v3 header spelling with the feed owner (CQ-I1)	tool:repo (`ingest.py`); org.data_model; feat.design (B1.3)
R-I4	TSK-02 reads the `paid_date` key that TSK-01 adds	Build TSK-01 first, then TSK-02, in the same lane	feat.tasks (section 1, TSK-02 "Depends on")

7. Hand-off to the next Data Engineering steps

- DE-05 (Data Model Changes): the registered Invoice already lists `paid_date` (date or None), added in feed v3. The new reject reason `'paid_date'` is the only addition to record (org.data_model; feat.design B1.3).
- DE-07 (Transformation): refines IN-3 and IN-7. Ingestion adds nothing more to change (sections 3 and 4).
- DE-08 (Coding): works from the file budget in section 4 and the rules in section 3.

8. Open questions (non-blocking)

ID	Question	Interim answer	Owner (proposed)
CQ-I1	Is the v3 header exactly `paid_date` (lower case, no spaces)? `parse_feed` matches the header text exactly (R-I3)	Yes, as in org.data_model and feat.design	Data engineering lead / feed owner
CQ-I2	Carried DQ-1: which production component fetches the feed and calls `pipeline.run` and `delays_by_vendor`?	Not needed for the ingestion code. The end-to-end harness wires the stages in this feature	Architect
CQ-I3	Carried OQ-B6: the requirement says `paid_date` is empty when unpaid (record:memory, journey requirement). Should an unpaid row that has a date be rejected?	Accepted and the date kept (feat.design DD-7; journey:feat.clarifications CL-DE5)	Product / Architect
CQ-I4	Should `data/sample_feed.csv` gain a v3 example? It is outside the task file budgets	No change in this feature (IN-9)	Data engineering lead

Carried unchanged: TQ-10 (reason precedence), DQ-4 (no `__all__` export), PQ-6 (feed v3 in production) (feat.design D3; feat.tasks section 7).

9. Validation check

Check	Result	Source
Existing sources and pipelines read	Pass: source, ingest, transform, data quality, pipeline, schedule, version, tests, CI, sample feed	tool:repo; file:ap_pipeline/ci.py; file:ap_pipeline/data/sample_feed.csv
Ingestion sources and pipelines to create or update listed	Pass: no new source or pipeline; `parse_feed` unchanged; additive update to `to_model` (IN-3); new `delays.py` (IN-7)	section 3
Minimum lines of code	Pass: one existing module updated, one new module, version step and tests; no change to ingest, data quality, pipeline or schedule	sections 3 and 4; feat.tasks (section 2 size budget)
Matches the registered Data Model	Pass: the record keys equal org.data_model's Invoice fields	org.data_model; feat.design (DC-1)
Secrets in all environments	Pass: none required, none granted, no gap	file:hobasa-data-engineering-agent/outputs/FEAT-ZD-48600/task.secrets_scope.md
Production or source-system data read	None. Only inputs, code and synthetic examples were used	record:brief
Conflict with an approved output	None	feat.tasks; feat.design; journey:feat.clarifications
Blocking questions	None	—